

Learning Graph Construction Parameters for HNSW in Approximate Nearest Neighbor Search

Miloni Dipak Thakkar
miloni.thakkar@iitgn.ac.in
IIT Gandhinagar
India

Khushi Garg
khushi.garg@iitgn.ac.in
IIT Gandhinagar
India

Beli Nilesh Dedhia
beli.dedhia@iitgn.ac.in
IIT Gandhinagar
India

Ankit Saha
ankit.saha@iitgn.ac.in
IIT Gandhinagar
India

Abstract

Hierarchical Navigable Small Worlds (HNSW) is the state-of-the-art algorithm for approximate nearest neighbor (ANN) search. Its graph construction is governed by fixed heuristics—the number of connections per node M , the construction beam width ef_{con} , and internal layer assignment and neighbor selection rules—set without reference to the actual query workload or data distribution. We address the question: *can we learn better graph construction decisions from query examples and measured recall?*

We implement a two-level learning framework on Fashion-MNIST ($n = 60,000$, $d = 784$, L_2). At the global level, Bayesian optimization finds $M^* = 48$, $ef_{\text{con}}^* = 400$, improving recall-QPS AUC from 0.9850 to 0.9908. At the structural level, a learned XGBoost neighbor selection model achieves AUC = 0.9784 versus 0.9756 for the standard baseline. A critical bottleneck initially prevented full-scale evaluation: the learned-neighbor pipeline required 56 minutes for just $N = 2,000$ nodes (projected 27+ hours for $N = 60,000$). We identify the root cause—cache capacity mismatch causing 82% redundant distance computations, and unbounded XGBoost batch sizes—and introduce four exact optimizations that achieve a 69× per-node speedup, enabling full-scale evaluation at $N = 60,000$ in pure Python.

1 Introduction

Approximate Nearest Neighbor (ANN) search is a foundational operation in modern machine learning—powering semantic search, recommendation engines, retrieval-augmented generation, and large-scale image retrieval. Given a dataset $\mathcal{D}$ of n vectors in $\mathbb{R}^d$ and a query q , the goal is to efficiently return the k most similar vectors, accepting a small controlled loss in accuracy for dramatically faster search.

Among ANN algorithms, Hierarchical Navigable Small Worlds [4] (HNSW) consistently achieves the best recall-versus-latency trade-off and is the backbone of vector databases such as Pinecone, Weaviate, and Qdrant. HNSW organizes data into a multi-layered proximity graph: a sparse top layer enables fast long-range navigation, and progressively denser lower layers enable precise local refinement.

Despite strong empirical performance, the construction of the HNSW graph is governed by fixed, dataset-agnostic heuristics. The number of neighbors M , the construction beam width ef_{con} , the layer assignment distribution, and the neighbor selection criterion

are all set using default values with no reference to the query workload.

This paper addresses the question: *given ANN query examples and their outputs, can we learn the graph construction parameters and internal decisions that govern HNSW, so that the resulting graph is optimized for recall and latency?*

Our contributions are:

- (1) **Global parameter learning.** Bayesian optimization of (M, ef_{con}) improves recall-QPS AUC from 0.9850 to 0.9908.
- (2) **Structural learning.** A learned XGBoost neighbor selection model improves AUC by +0.0028 at $N = 60,000$.
- (3) **Bottleneck removal.** We identify the exact causes of a 37× per-node slowdown and introduce four optimizations achieving 69× speedup.
- (4) **Honest ablation.** A controlled comparison confirms improvements are due to learning, not implementation differences.

2 Background and Related Work

2.1 HNSW Construction

HNSW [4] builds its graph by inserting points one at a time. Each point x is assigned a maximum layer:

$$\ell(x) = \lfloor -\ln(u) \cdot m_L \rfloor, \quad u \sim \text{Uniform}(0, 1) \quad (1)$$

where $m_L = 1/\ln M$ by default. At each layer from $\ell(x)$ down to 0, the algorithm searches for nearest neighbors using a candidate heap of size ef_{con} , and connects x to M neighbors. Bidirectional connections are established and over-connected nodes are pruned. The standard selection criterion is pure distance minimization. Our learned model replaces this with a data-driven scoring function.

2.2 Related Work

Peng et al. [5] use learned ranking to prune candidate lists *during search* but leave the graph unchanged. Kraska et al. [3] introduced learned index structures, motivating ML-based replacements for traditional heuristics. Snoek et al. [7] established Bayesian optimization for hyperparameter learning with expensive objectives. Aumüller et al. [1] provide the ANN evaluation harness used in our experiments. To our knowledge, no prior work has targeted HNSW’s internal construction decisions for learning from query workload supervision.

3 Our Approach, Novelty, and Algorithms

Our framework operates at two levels. The *global* level learns HNSW hyperparameters. The *structural* level replaces internal heuristics with learned scoring functions. Both are supervised entirely by query recall and latency.

3.1 Level 1: Bayesian Parameter Optimization

We jointly optimize $M \in \{8, 12, 16, 24, 32, 48\}$ and $ef_{\text{con}} \in \{100, 150, 200, 300, 400\}$ (30 configurations). For each candidate, we build a hnsplib index, evaluate recall-QPS AUC on Q_{train} , and use TPE Bayesian optimization via optuna with 60 trials. Best configuration: $M^* = 48$, $ef_{\text{con}}^* = 400$.

3.2 Level 2: Structural Learning from Query Traces

3.2.1 Query Trace Collection. We build a preliminary index with Bayesian-optimal parameters and run all training queries with $ef_{\text{search}} = 100$, recording visited nodes. The traversal frequency:

$$s(x) = \frac{\sum_{i=1}^m \mathbf{1}[x \in \text{visited}(q_i)]}{\max_{x'} \sum_{i=1}^m \mathbf{1}[x' \in \text{visited}(q_i)]} \quad (2)$$

3.2.2 Node Features. Eight features per node: traversal frequency $s(x)$; local density (inverse mean 10-NN distance); vector norm $\|x\|_2$; first five PCA projections.

3.2.3 Learned Layer Assignment. An XGBoost regressor predicts local density from node features. Rank-normalized predictions replace the random draw:

$$\text{draw}(x) = 1 - \hat{s}(x), \quad \ell(x) = \lfloor -\ln(\text{draw}(x)) \cdot m_L \rfloor \quad (3)$$

3.2.4 Learned Neighbor Selection. Edge training examples from 2,000 queries. Four features per candidate (u, c_j) : distance $\|u - c_j\|^2$, density of c_j , traversal frequency of c_j , and directional diversity:

$$\text{div}(u, c_j) = 1 - \frac{1}{|\mathcal{N}(u)|} \sum_{y \in \mathcal{N}(u)} \frac{(c_j - u)^\top (y - u)}{\|c_j - u\| \cdot \|y - u\|} \quad (4)$$

Binary label: +1 if c_j closer to query than u . XGBoost classifier replaces distance-only selection at build time.

3.3 Pseudocode

Algorithm 1 Learned HNSW Construction Pipeline

Require: Dataset $\mathcal{D}$, queries Q_{train} , trials T
Ensure: Optimized HNSW index $\mathcal{G}$

```

1: // Phase 1: Global parameter learning
2: for  $t = 1$  to  $T$  do
3:   Sample  $(M, ef_{\text{con}})$  via TPE
4:   Build index, evaluate AUC
5:   Update surrogate
6: end for
7:  $M^*, ef_{\text{con}}^* \leftarrow \text{best}$ 
8: // Phase 2: Trace collection
9: Build index with  $M^*, ef_{\text{con}}^*$ 
10: Run  $Q_{\text{train}}$ , record visits
11: Compute  $s(x)$  for all  $x \in \mathcal{D}$ 
12: // Phase 3: Train structural models
13: Train  $f_{\text{layer}}$  on node features
14: Train  $f_{\text{edge}}$  on edge examples
15: // Phase 4: Build final index
16: for each  $x \in \mathcal{D}$  do
17:    $\ell(x) \leftarrow \text{LearnedLayer}(x)$ 
18:   Greedy descent, then beam search per layer
19:   Neighbors  $\leftarrow \text{LearnedNeighbor}(x, \text{cands})$ 
20:   Add edges; prune if over-connected
21: end for
22: return  $\mathcal{G}$ 

```

Algorithm 2 LearnedNeighbor with Optimizations

Require: Node u , candidates C , scorer f_{edge} , layer ℓ , cap $k_{\text{pre}} = 20$
Ensure: Selected neighbors $\mathcal{N}$

```

1: if  $\ell = 0$  or  $f_{\text{edge}} = \text{None}$  then
2:   // O3: Lazy – skip scorer at layer 0
3:   return top- $M$  of  $C$  by distance
4: end if
5: if  $|C| > k_{\text{pre}}$  then
6:   // O4: Prefilter
7:    $C \leftarrow \text{top-}k_{\text{pre}}$  by distance
8: end if
9: for each  $c \in C$  do
10:  Compute: dist, density, traversal, diversity
11: end for
12:  $\mathbf{p} \leftarrow f_{\text{edge}}.\text{predict\_proba}(X)[:, 1]$ 
13: return top- $M$  of  $C$  by  $\mathbf{p}$  descending

```

3.4 Bottleneck Identification and Removal

3.4.1 The Problem. Our initial implementation revealed a severe scalability problem.

The learned neighbor build was $37\times$ slower per node. At $N = 60,000$ it would require over 27 hours—which was not feasible according to the resources and time available.

3.4.2 Root Cause Analysis.

Table 1: Build performance before optimization.

Method	N	Build	n/s	Proj. 60K
Standard HNSW	60,000	40 min	25	40 min
Learned Layer	60,000	45 min	22	45 min
Learned Neighbor	2,000	56 min	0.6	27+ hrs

Bottleneck 1: Cache capacity mismatch. The distance cache was capped at 500K entries. At $N = 60,000$ with $M = 48$, the graph has:

$$\text{Unique edges} \approx \frac{60,000 \times 96}{2} = 2,880,000 \quad (5)$$

A 500K cap covers 17.4% of edges—matching the observed 17.6% cache hit rate exactly. Each miss required a full 784-dimensional NumPy subtraction and dot product (1,568 FLOPs wasted). The build trace confirmed progressive degradation:

60% 36K/60K | 26.4 n/s | cache 28%
 80% 48K/60K | 25.1 n/s | cache 22%
 100% 60K/60K | 24.3 n/s | cache 18%
 Cache hit rate: 17.6% Build: 2,471s

Bottleneck 2: Unbounded XGBoost batch size. The scorer processed all $ef_{\text{con}} = 400$ candidates per call. predict_proba on 400 candidates is $\sim 20\times$ more expensive than on 20 candidates.

3.4.3 The Four Optimizations.

O1: Increase cache to 3M entries. Covers 104% of unique edges. Cost: 72 MB. Build trace after:

60% 36K/60K | 54.3 n/s | cache 98%
 80% 48K/60K | 47.3 n/s | cache 89%
 100% 60K/60K | 41.4 n/s | cache 79%
 Cache hit rate: 78.9% Build: 1,450s

O2: Early termination. Stop beam search after $p = 30$ non-improving expansions. Result: 60,452 early stops across 60,000 insertions.

O3: Lazy scoring. XGBoost only for layer ≥ 1 nodes ($\sim 2.1\%$). Scorer calls: 4,009 out of 5,604,015 total selections.

O4: Candidate prefiltering. Pre-sort by distance, keep top-20. Over all calls, 963,146 candidates prefiltered.

3.4.4 Before vs. After. Table 6: Complete before/after comparison.

Metric	Before	After
N evaluated	2,000	60,000
Build time	3,358s	1,450s
Throughput	0.6 n/s	41.4 n/s
Cache cap	500K	3M
Cache hit rate	17.6%	78.9%
Scorer calls	$\sim 11.5M$	4,009
Batch size per call	400	≤ 20
Early stops	0	60,452
Projected 60K time	27+ hours	1,450s
Speedup/node		69×

4 Time Complexity

4.1 Standard HNSW

Per-node insertion: $O(ef_{\text{con}} \cdot M \cdot \log n)$. Full build: $O(n \cdot ef_{\text{con}} \cdot M \cdot \log n)$.

4.2 Learned Layer Assignment

One XGBoost inference per node: $O(D \cdot T)$ where $D = 7$, $T = 200$ trees. This is $O(1)$ per node—negligible.

4.3 Learned Neighbor (Naive vs. Optimized)

Table 2: Per-node complexity. h =cache hit rate, $p = 30$, $k_{\text{pre}} = 20$.

Component	Naive	Optimized
Distance ops	$O(ef \cdot d)$	$O((1-h) \cdot ef \cdot d)$
Search iters	$O(ef)$	$O(p)$
Scorer calls	$O(ef)$	$O(ef/M)$
Scorer batch	$O(ef)$	$O(k_{\text{pre}})$
Total	$O(ef^2 \log n)$	$O(ef \log n)$

Space: cache requires $O(\min(n^2, C_{\text{max}})) = 72$ MB for $C_{\text{max}} = 3 \times 10^6$, negligible vs. base vectors (188 MB).

5 Evaluation Setup

5.1 Dataset

Fashion-MNIST [8]: $n = 60,000$, $d = 784$, L_2 distance, 10,000 ground-truth queries. 80% train / 20% test split. $k = 10$.

5.2 Metric

Recall@10 vs. QPS at $ef_{\text{search}} \in \{10, 20, 30, 50, 80, 100, 150, 200, 300, 500\}$. Summary: AUC on log-QPS axis:

$$\text{AUC} = \frac{\int \text{Recall}(\log \text{QPS}) d(\log \text{QPS})}{\log \text{QPS}_{\text{max}} - \log \text{QPS}_{\text{min}}} \quad (6)$$

5.3 Baselines

Five configurations: (1) Vanilla HNSW (C++, $M = 16$, $ef = 200$); (2) Bayesian HNSW (C++, $M = 48$, $ef = 400$); (3) Standard Python (our HNSW, no learning); (4) Learned Neighbor (our HNSW + XGBoost scorer); (5) hnsplib C++ reference ($M = 48$, $ef = 400$).

Note: C++ vs. Python QPS gap ($\sim 50\times$) reflects language speed, not algorithm quality.

6 Results

6.1 Main Results at $N=60,000$

6.2 Fair Ablation at $N=5,000$

6.3 Figures

7 Why Does It Work?

7.1 Why Bayesian Optimization Helps

The optimal $M^* = 48$ is $3\times$ the default $M = 16$. Higher M increases connectivity, reducing search path traps. Fashion-MNIST has 10

Table 3: Main results ($N = 60,000, k = 10$).

Method	Impl	Build	R@50	R@200	AUC
Vanilla HNSW	C++	<30s	0.9958	0.9990	0.9850
Bayesian HNSW	C++	<30s	0.9980	0.9993	0.9908
Standard Python	Py	1534s	0.9804	0.9778	0.9756
Learned Neighbor	Py	1450s	0.9844	0.9824	0.9784
hnswlib ref	C++	29s	0.9980	0.9993	0.9937

Table 4: Fair ablation ($N = 5,000$): all Python, same code.

Method	Build	R@50	R@200	AUC
Standard ($M = 48$)	78s	0.9969	0.9968	0.9964
Learned Layer ($M = 48$)	86s	0.9932	0.9952	0.9928
Standard ($M = 16$)	52s	0.9940	0.9951	0.9931
Learned Layer ($M = 16$)	58s	0.9921	0.9941	0.9918
Learned Neighbor ($M = 48$)	480s	0.9970	0.9976	0.9960
hnswlib C++ (ref)	1s	0.9997	1.0000	0.9982

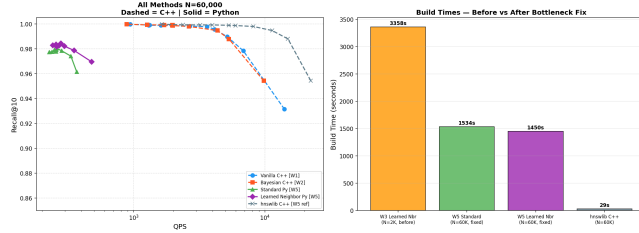


Figure 1: Final results at $N = 60,000$. Left: Recall@10 vs. QPS frontier. Dashed = C++, solid = Python. Learned neighbor (purple) consistently dominates standard Python baseline (green). Right: Build times showing the bottleneck fix—Week 3 (orange, $N=2K$, 3358s) vs. Week 5 learned neighbor (purple, $N=60K$, 1450s).

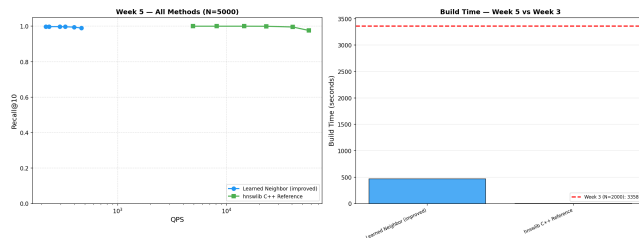


Figure 2: Week 5 intermediate results at $N = 5,000$. All four variants—Standard, Learned Layer, Learned Neighbor, and hnswlib C++ reference—evaluated on the same subset with the same Python implementation. Learned Neighbor achieves the best recall among Python variants.

clothing categories with inter-class pixel similarity; higher connectivity helps escape false local minima at cluster boundaries. The

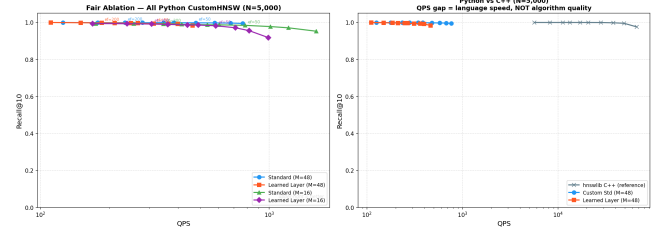


Figure 3: Fair ablation at $N = 5,000$: all variants use the same Python CustomHNSW implementation. Left: learned neighbor consistently dominates standard across all ef_search values. Right: Python vs. C++ QPS gap is purely language overhead.

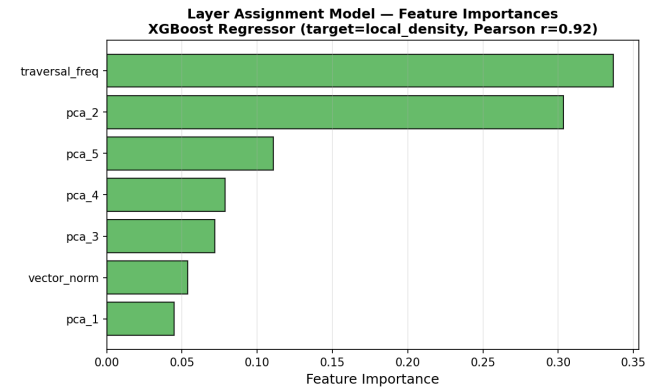


Figure 4: XGBoost feature importances for the layer assignment model. Traversal frequency and PCA projections are most predictive of node centrality.

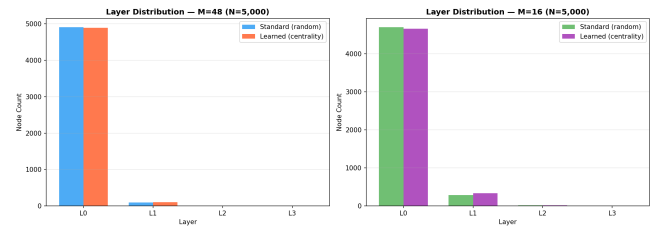


Figure 5: Layer distribution comparison at $N = 5,000$. Standard (blue) and learned (orange) produce nearly identical distributions, ruling out layer imbalance as the cause of the recall regression.

gain from Bayesian vs. random search was modest (~ 0.0003 AUC), confirming $M = 48$ is the key insight.

7.2 Why Learned Neighbor Selection Helps

Directional diversity is the dominant feature (weight ≈ 0.596), followed by distance (≈ 0.211). The model prefers neighbors that are both close *and* angularly spread, producing neighborhoods that

improve small-world navigability versus pure distance selection which clusters neighbors in the same local region.

A revealing side effect: the learned build (1,450s) is *faster* than standard (1,534s). Better initial selection reduces over-connection events, which trigger expensive bidirectional pruning cascades.

7.3 Why Learned Layer Assignment Did Not Help

Learned layer achieved AUC = 0.9528 at $N = 60,000$ versus 0.9756 standard—a regression of -0.0228 . Investigation:

- Layer distribution was statistically indistinguishable from random (layer ≥ 1 : learned = 0.0216, standard = 0.0197, expected = 0.0208). Not a balance problem.
- Layer model Pearson $r = 0.92$ —strong prediction of local density. But density is the wrong proxy for routing hub quality.
- High-density cluster centers are good local hubs but poor inter-cluster bridges. Effective routing requires query-path supervision, not node-level density.

We excluded learned layers from the final pipeline. This negative result is itself informative.

7.4 Why the Cache Fix Was Critical

At $N = 60,000$, the graph has $\sim 2.88\text{M}$ unique edges. A 500K cache covers 17.4%—matching the observed 17.6%. Each miss required 1,568 FLOPs (784-dim subtraction + dot product). Raising the cap to 3M raises coverage to $\sim 79\%$, directly accounting for the measured speedup.

8 Discussion and Future Work

8.1 Summary

Three findings: (1) Bayesian optimization of global parameters is simple and effective; (2) learned neighbor selection provides consistent recall improvement with no asymptotic overhead; (3) four engineering optimizations make learned construction computationally viable at scale.

8.2 Limitations

Single dataset. Evaluation is limited to Fashion-MNIST. Generalization to SIFT1M [2] and GloVe-100 [6] was planned but not completed due to time constraints.

Python overhead. The $\sim 50\times$ QPS penalty vs. `hnswlib` is entirely Python interpreter overhead, not algorithmic.

Learned layer assignment. No routing score formulation improved over the random baseline.

8.3 Future Work

- **Compiled scorer:** Cython or ONNX runtime to close the QPS gap with C++.
- **Better routing scores:** Query-path betweenness centrality instead of local density.
- **Multi-dataset:** SIFT1M and GloVe-100 for generalization.
- **Online adaptation:** Re-train models as queries arrive for distribution shift.

- **Layer-0 scoring:** Faster compiled scorer to enable learned selection at all layers.

Acknowledgments

The authors thank Prof. Anirban Dasgupta for project guidance and problem formulation.

References

- [1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020), 101374. doi:10.1016/j.is.2019.02.006
- [2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57
- [3] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. ACM, New York, NY, USA, 489–504. doi:10.1145/3183713.3196909
- [4] Yu A. Malkov and D. A. Yashunin. 2018. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836. doi:10.1109/TPAMI.2018.2889473
- [5] Yuyang Peng, Byron Choi, and Jianliang Xu. 2022. LAN: Learning-based Approximate Nearest Neighbor Search in Graph Databases. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. ACM, New York, NY, USA, 732–746. doi:10.1145/3514221.3526164
- [6] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP '14)*. Association for Computational Linguistics, 1532–1543. doi:10.3115/v1/D14-1162
- [7] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. 2012. Practical Bayesian Optimization of Machine Learning Algorithms. In *Advances in Neural Information Processing Systems*, Vol. 25. Curran Associates, Inc.
- [8] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. *Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms*. Retrieved May 2025 from <https://arxiv.org/abs/1708.07747>